Trajectory – Learning & Engineering Roadmap

A comprehensive guide to the technologies, frameworks, infrastructure setups, and architectural design patterns to master while building and reviewing the Career Operating System project.

Phase 0 – Already Know (The Baseline)

- **Java Foundations:** Core syntax, collections framework, exception handling, and robust Object-Oriented Programming (OOP) principles.
- **Web Development Basics:** Core understanding of HTML, CSS, and general RESTful API communication paradigms.
- **Version Control:** Fundamental Git operations including branch creation, merging conflicts, and managing pull requests.
- **Basic React:** Core understanding of declarative UI rendering, functional components, and element rendering cycles.

Phase 1 – Core Full-Stack Extension

- **TypeScript:** Transitioning from JavaScript to strict type safety. Master types, interfaces, custom enums, generics, and interface shapes for API network responses.
- **Advanced React & Component Design:** Deep dive into React Hooks (useState, useEffect, useMemo, useCallback, useRef) alongside managing component architecture patterns (Pages → Features → Layouts → Reusable Shadcn UI primitives).
- **Modern Enterprise Java:** Implementing modern Java 21 features, heavily utilizing Java Records for lightweight, immutable Data Transfer Objects DTOs and switch expressions.

Phase 2 – Project Specific & Domain Logic

- **Spring Boot 3.x Ecosystem:** Setting up production-grade MVC Web layers, wiring bean dependency injections, and building global exception interceptors using @RestControllerAdvice.
- **Relational Data Persistence:** Mastering Spring Data JPA and Hibernate for multi-tenant data access, working extensively with UUID primary and foreign key strategies.
- **Database Version Control:** Implementing Flyway migration histories to manage incremental PostgreSQL schema states seamlessly within localized repository structures.

Phase 3 – AI Integration Layer

- **Spring AI Framework:** Using a standardized framework layer to orchestrate Large Language Models (LLMs) rather than utilizing raw, vendor-specific HTTP client scripts.
- **Structured Output Engineering:** Utilizing BeanOutputConverter and StructuredOutputConverter along with precise prompt engineering templates to guarantee strict JSON output serialization into strongly typed Java Records.

- **Resiliency & Provider Routing:** Configuring OpenAI-compatible chat clients to hit the ultra-low latency Groq Cloud API (Llama 3/Mixtral) with automated fallback paths executing over Google Gemini Pro via circuit-breaker design patterns.

Phase 4 – State & Client-Side Architecture

- **Multi-Page Client-Side Routing:** Mastering React Router (v6+) to engineer a decoupled, single-page application layout. Understanding how to wrap routes dynamically using BrowserRouter, Routes, and nested Route contexts.
- **The Layout Shell Pattern:** Implementing static shell wrappers utilizing the <Outlet /> component to persist global interactive UI regions (like your responsive, glassmorphic navigation sidebar) while dynamically swapping inner child views based on URL parameter strings.
- **Dynamic Route Parameters:** Architecting type-safe, rest-parameter endpoints (such as /applications/:id) to extract specific resource identifiers from the URL string, fueling localized database calls and rendering the chronological application timeline view seamlessly.
- **Server-State Management:** Implementing TanStack Query (React Query) on the frontend to gracefully separate remote asynchronous backend caching, data fetching, error metrics, and mutations from local screen rendering cycles.
- **High-Density Render Optimization:** Utilizing Zustand UI state boundaries to dynamically toggle between a standard card layout and a compressed tabular grid framework, minimizing active DOM node counts when managing high-volume data arrays.
- **Lightweight Client-State Store:** Using Zustand as a high-speed, minimal memory global workspace configuration engine for ephemeral UI values, active modal flags, and user visual themes.
- **Optimistic UI Syncing:** Designing the layout layer to instantly alter visual states upon mutation events (e.g., pipeline status drag-and-drops) while spinning background HTTP operations, featuring comprehensive local cache rollback hooks in case backend nodes drop out.

Phase 5 – Local Infrastructure & Background Daemons

- **Docker & Multi-Container Composites:** Orchestrating localized microenvironments containing independent PostgreSQL 16 databases, Redis instances, and MinIO storage systems through docker-compose.yml healthcheck parameters.
- **S3-Compatible Binary Storage Vaults:** Handling high-speed multi-part file streams natively into MinIO / Supabase Storage engines, using partitioned user folder structures (vault/users/{id}/resumes/) to isolate sensitive data objects safely.
- **Automated Scheduling (Cron Daemons):** Building independent, nonblocking asynchronous server background tasks using Spring's @Scheduled annotations to scan temporal dates daily and automatically transition stale pipelines to GHOSTED or auto-archive them to WITHDRAWN states based on explicit user-configured boundary thresholds.

Phase 6 – Containerization & Cloud Deployment Architecture

- **Dockerization Strategy:** Writing multi-stage Dockerfile configurations. The backend relies on a lightweight Eclipse Temurin JDK runtime image to minimize artifact size, while the frontend utilizes a multi-stage Node build that copies static assets directly into a production-grade Nginx alpine image.

- **AWS Core Services Integration:**
  - **Compute:** Deploying containerized services using AWS ECS (Elastic Container Service) with AWS Fargate for serverless container execution, eliminating manual EC2 instances overhead.
  - **Database & Cache:** Swapping local Docker configurations for production-ready managed cloud systems—migrating PostgreSQL data to AWS RDS (Relational Database Service) and local Redis workloads to AWS ElastiCache.
  - **Storage & Network:** Migrating local MinIO buckets natively to AWS S3 using identity-based access controls, and setting up an AWS VPC (Virtual Private Cloud) with public/private subnets to completely isolate backend data tiers from the public internet.
- **CI/CD GitOps Pipeline:** Configuring GitHub Actions to automate production pipelines. Every push to the target branch triggers an automated workflow that runs structural JUnit tests, logs into AWS ECR (Elastic Container Registry), builds and packages production Docker images, and updates the ECS Task Definition to execute an automated rolling deployment.

Phase 7 – Core Engineering Concepts Applied

- **Separation of Concerns & Decoupled Architecture:** Isolating business tasks cleanly by splitting the software system into distinct project layers: Presentation (React SPA), Application Framework (Spring Boot Web controllers), and Data Access (JPA/PostgreSQL).
- **Stateless Security Architectures:** Securing endpoints cleanly via Spring Security using asymmetrical, short-lived JSON Web Tokens (JWT) paired with distributed Redis token-bucket rate limiters to prevent token abuse.
- **Audit Trail Timelines:** Enforcing strict data consistency rules where status mutations instantly trigger asynchronous data hooks that append historical verification entries directly into tracking models (application_status_history).

Interview Focus Points (How to talk about this project)

- **Explain the E2E Data Pipeline:** Be prepared to articulate exactly how a text block strings from a frontend form, routes via an optimistic query state hook into an encrypted API endpoint, processes through Spring AI prompt structures over Groq/Gemini, maps inside Java Records, saves to a PostgreSQL relational layout, and triggers automatic status logging rows.
- **Explain the Local-to-Cloud Infrastructure Transition:** Be ready to articulate how you decouple infrastructure between development and production. "Locally, I use Docker Compose to run PostgreSQL, Redis, and MinIO to minimize costs and speed up agent loops. In production on AWS, I shift to managed services like RDS, ElastiCache, and native S3 via IAM roles, ensuring scalability, high availability, and strict network isolation without modifying the core Spring Boot application logic."
- **Defend the Architectural Decisions:** Explain why a monorepo layout was utilized, why UUIDs were preferred over standard integer sequences for APIs, why server-state was decoupled from client-state via React Query, and how virtual threads protect system processing limits when external AI frameworks throttle response performance.